

**BACHELOR OF COMPUTER SCIENCE
SCHOOL OF COMPUTER SCIENCE
BINA NUSANTARA UNIVERSITY
JAKARTA**

ASSESSMENT FORM

Course: COMP6047001 – Algorithms and Programming

Method of Assessment: Case Study

Semester/Academic Year: 1/2022–2023

Name of Lecturer : Indra Dwi Rianto, S.Kom., S.Si., M.T.I.

Date : 19 Januari 2023

Class : LC01

Topic : Material Review II

Group Members:	1	Jonathan Alvindo Fernandi
	2	_____
	3	_____
	4	_____
	5	_____
	6	_____
	7	_____
	8	_____

Student Outcomes:

(SO 2) Mampu merancang, mengimplementasikan, dan mengevaluasi solusi berbasis komputasi untuk memenuhi serangkaian persyaratan komputasi dalam konteks ilmu komputer.

Able to design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of computer science.

Learning Objectives:

(LO 2.2) Mampu mengimplementasikan solusi berbasis komputasi untuk memenuhi serangkaian persyaratan komputasi tertentu dalam konteks ilmu komputer.

Able to implement a computing-based solution to meet a given set of computing requirements in the context of computer science.

No.	Assessment Criteria	Weight	Excellent (85–100)	Good (75–84)	Average (65–74)	Poor (0–64)	Score	(Score × Weight)
1	Ability to identify the problems and explain the solution.	25%	The problem is well defined and solution is clearly explained and detailed.	The problem is well defined and solution is less clearly explained.	The problem is defined and solution is less clearly explained.	The problem and solution are badly defined and explained respectively.	100	25
2	Apply logical thinking.	25%	Correctly and effectively applying logic thinking to solve the problem.	Correctly applying logic thinking to solve the problem.	Partially correct applying logic thinking to solve the problem.	Incorrectly applied the logic thinking.	100	25
3	Ability to construct a C program.	25%	All the syntaxes in the program are correctly and effectively applied.	All the syntaxes in the program are correctly applied.	Only some of the syntax in the program are correctly applied.	None of the syntax in the program are correctly applied.	100	25
4	Ability to choose the appropriate algorithm	25%	Correctly choosing the most effective algorithm to solve the problem.	Correctly choosing effective algorithm to solve the problem.	Correctly choosing the algorithm to solve the problem.	Incorrectly choosing the algorithm to solve the problem.	100	25
Total Score: $\sum(\text{Score} \times \text{Weight})$								100

Remarks:

ASSESSMENT METHOD

Instructions

1. This is an individual assignment and will be held in a review topic session with a duration of 1 week, or week 13.
2. You will be given 3 questions. The questions are case studies.
3. There will be 4 files that you need to upload (and zipped):
 - a. One is a Word file. This file consists of your explanation of the solution. The explanation should be given using a flowchart for better explanation. A correct and detailed flowchart will give a higher score.
 - b. Another 3 are .c files of your solution which will be checked by the respective lecturers.
4. Do not cheat.

Note for Lecturers:

1. This case study assignment will be held for a duration of 1 week during the review topic, or week 13.
2. The answer is manually checked by each lecturer (not by the system).
3. You may refer to the rubric table given above.

Questions

You are required to write a program that can read a file and perform several functions. The required file can be downloaded from the link:

<https://1drv.ms/u/s!AhuAx03LAKWtnOM9O1wIXSAR84Z67g?e=IVmH5x> .

File description:

This file is a CSV file; therefore, each column is separated by a comma. This file consists of 3939 rows of Housing Data in Malaysia with no missing values for each row. This file also has a **header**. **Therefore, when your program loads the data, your program should be able to skip this header before passing the data into your record variable.** A glimpse of the data:

```
AoL > file.csv
1 Location 1,Location 2,Price,Rooms,Bathrooms,CarParks,Type,Area,Furnish
2 Mont-Kiara,Kuala-Lumpur,1000000,2,2,0,Built-up,1000,Partly
3 Cheras,Kuala-Lumpur,310000,3,2,0,Built-up,1000,Partly
4 Kepong,Kuala-Lumpur,358000,3,3,0,Built-up,1000,Partly
5 Taman-Desa,Kuala-Lumpur,455000,2,2,0,Built-up,1000,Partly
6 Kepong,Kuala-Lumpur,358000,3,3,0,Built-up,1000,Partly
7 Kepong,Kuala-Lumpur,358000,3,3,0,Built-up,1000,Partly
8 Bukit-Jalil,Kuala-Lumpur,505000,3,2,0,Built-up,1000,Partly
9 Jalan-Klang-Lama,Kuala-Lumpur,410000,3,2,0,Built-up,1000,Partly
10 Setapak,Kuala-Lumpur,278000,3,2,0,Built-up,1000,Partly
11 Sentul,Kuala-Lumpur,688000,3,2,0,Built-up,1000,Fully
12 Mont-Kiara,Kuala-Lumpur,660000,2,2,0,Built-up,1000,Fully
13 Jalan-Klang-Lama,Kuala-Lumpur,338000,3,2,0,Built-up,1000,Partly
```

You are required to perform 3 functions as follows:

1. (40 Points) Describe

This function explains the information from each column. When running this function, **give a prompt to get input from the user** as the name of the column you want to describe. Then, display:

- a. For column loc1, loc2, room, bathrooms, carpark, type, or furnish, display:

- i. Frequency for each unique value
- ii. Maximum frequency
- iii. Minimum frequency

For example, if we call the describe function followed by loc1 as the column name, the program should display like this:

```
Batu-Caves : 19
Bangsar-South : 40
Kuchai-Lama : 37
Jinjang : 9
Bandar-Tasik-Selatan : 10
OUG : 40
Setiawangsa : 21
Sri-Hartamas : 63
Ampang-Hilir : 29
Seputeh : 21
Pandan-Indah : 10
Mid-Valley-City : 9
Brickfields : 19
Damansara : 5
Gombak : 3
Pandan-Jaya : 5
Alam-Damai : 1
Sunway-SPK : 8
Pandan-Perdana : 7
Other : 2
Happy-Garden : 1
Taman-Sri-Keramat : 1
TAMAN-MELATI : 1
Jalan-Sultan-Ismail : 12
Maximum value: Kepong with frequency: 450
Minimum value: TAMAN-MELATI with frequency: 1
```

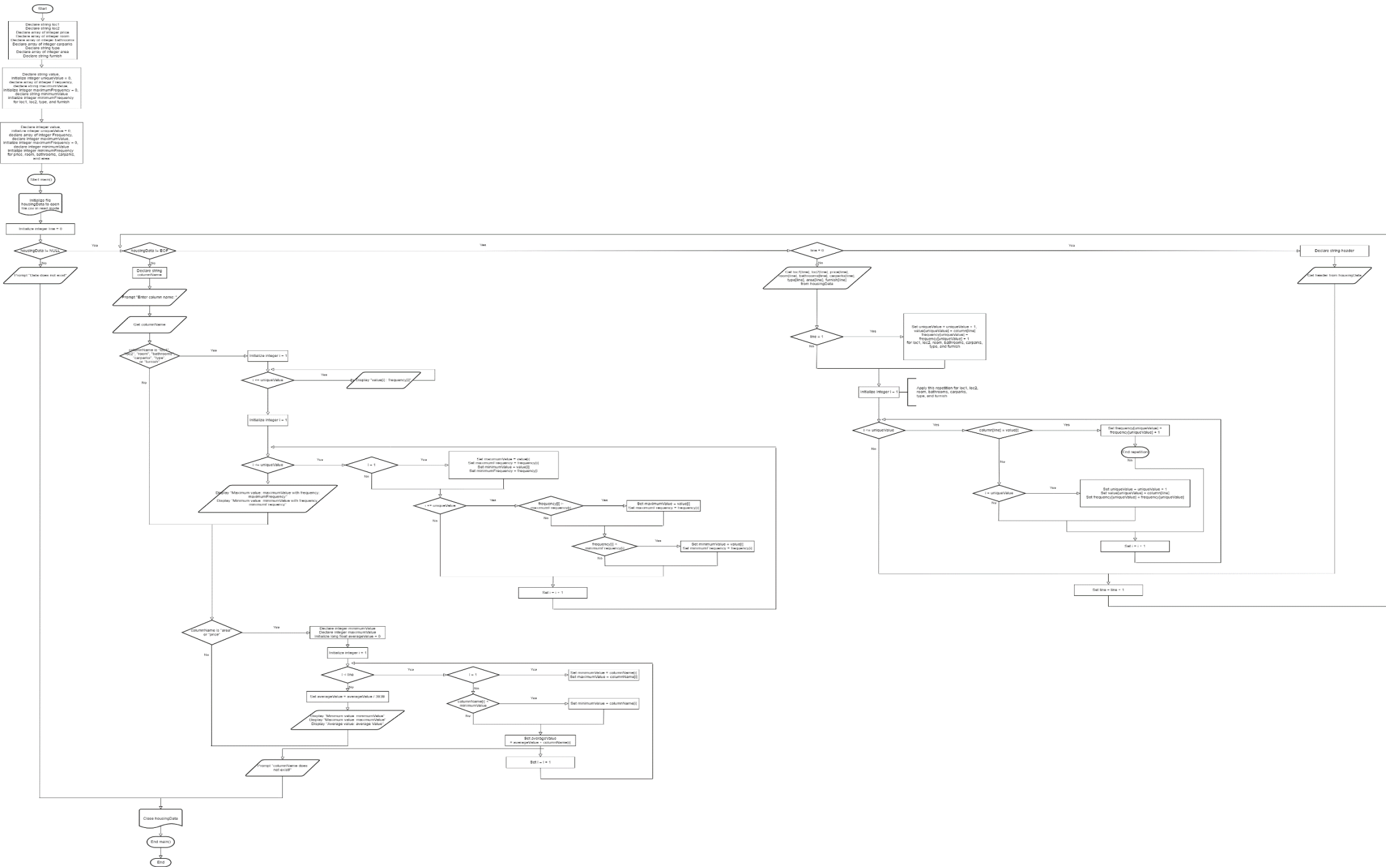
b. For the area and price columns, display only:

- i. Minimum value
- ii. Maximum value
- iii. Average value

Because area and price values are not discrete, there is no need to perform a frequency check for each of them.

Draw the Flowchart of your solution.

Explanation:



2. (40 Points) Search Data

To be able to handle search function, ask user to give input with format:

```
DataX in ColumnName
```

Your program should be able to parse above input. **It is prohibited to use 3 string input.** Then, display all data that has that DataX.

For example:

```
Partly in furnish
```

The above command will result in:

Kepong	Kuala-Lumpur	357000	3	2	0	Built-up	973	Partly
Salak-Selatan	Kuala-Lumpur	248000	3	2	0	Built-up	973	Partly
Mont-Kiara	Kuala-Lumpur	1160000	2	2	0	Built-up	973	Partly
City-Centre	Kuala-Lumpur	800000	3	2	0	Built-up	975	Partly
Sri-Petaling	Kuala-Lumpur	390000	3	2	0	Built-up	975	Partly
KLCC	Kuala-Lumpur	1400000	2	2	0	Built-up	976	Partly
KLCC	Kuala-Lumpur	1400000	2	2	0	Built-up	976	Partly
KLCC	Kuala-Lumpur	1400000	2	2	0	Built-up	976	Partly
KLCC	Kuala-Lumpur	1400000	2	2	0	Built-up	976	Partly
KLCC	Kuala-Lumpur	1400000	2	2	0	Built-up	976	Partly
Ampang	Kuala-Lumpur	690822	2	2	0	Built-up	977	Partly
Ampang	Kuala-Lumpur	690822	2	2	0	Built-up	977	Partly
Ampang	Kuala-Lumpur	690822	2	2	0	Built-up	977	Partly
Ampang	Kuala-Lumpur	690822	2	2	0	Built-up	977	Partly
Jalan-Ipoh	Kuala-Lumpur	638000	3	2	0	Built-up	977	Partly
Jalan-Ipoh	Kuala-Lumpur	486000	3	2	0	Built-up	978	Partly
Jalan-Ipoh	Kuala-Lumpur	486000	3	2	0	Built-up	978	Partly
Jalan-Ipoh	Kuala-Lumpur	486000	3	2	0	Built-up	978	Partly
Jalan-Ipoh	Kuala-Lumpur	540000	3	2	0	Built-up	978	Partly
Jalan-Ipoh	Kuala-Lumpur	540000	3	2	0	Built-up	978	Partly
Jalan-Ipoh	Kuala-Lumpur	486000	3	2	0	Built-up	978	Partly
Jalan-Ipoh	Kuala-Lumpur	486000	3	2	0	Built-up	978	Partly
Jalan-Ipoh	Kuala-Lumpur	486000	3	2	0	Built-up	978	Partly
Jalan-Ipoh	Kuala-Lumpur	486000	3	2	0	Built-up	978	Partly
Jalan-Klang-Lama	Kuala-Lumpur	755000	3	2	0	Built-up	978	Partly
KLCC	Kuala-Lumpur	760000	2	1	0	Built-up	980	Partly

However, your program also should be able to search using only a substring, for example:

pong in loc1

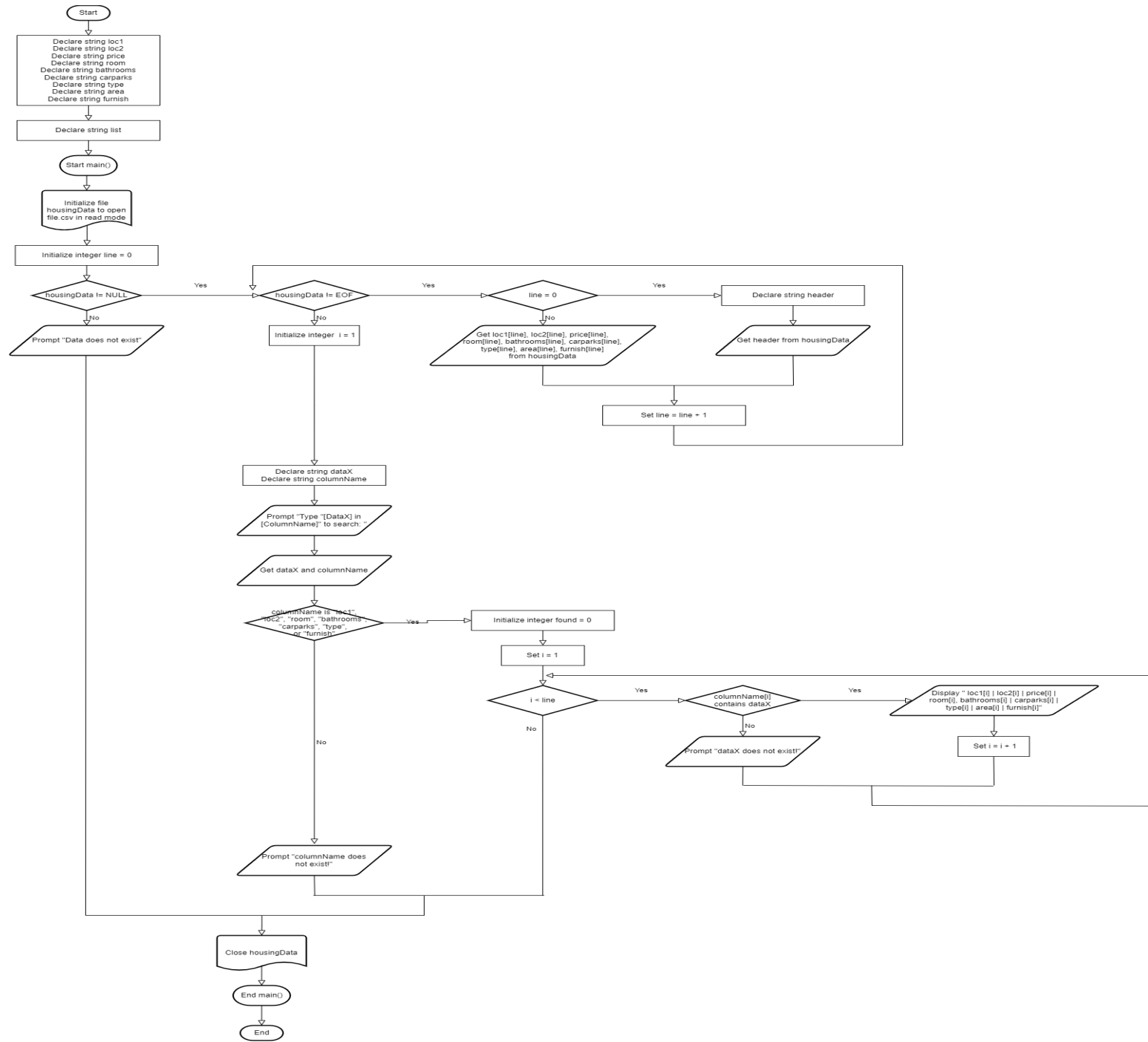
will result in:

Kepong	Kuala-Lumpur	2250000	5	4	0	Land-area	4500	Partly
Kepong	Kuala-Lumpur	930000	6	4	0	Land-area	4130	Partly
Kepong	Kuala-Lumpur	3400000	6	6	0	Land-area	4800	Unfurnished
Kepong	Kuala-Lumpur	398000	3	2	0	Built-up	630	Partly
Kepong	Kuala-Lumpur	465000	3	3	0	Land-area	630	Partly
Kepong	Kuala-Lumpur	365000	3	2	0	Built-up	630	Partly
Kepong	Kuala-Lumpur	480000	3	2	0	Land-area	630	Partly
Kepong	Kuala-Lumpur	350000	3	2	0	Built-up	630	Partly
Kepong	Kuala-Lumpur	480000	3	2	0	Land-area	630	Partly
Kepong	Kuala-Lumpur	4800000	5	6	0	Land-area	6466	Partly
Kepong	Kuala-Lumpur	200000	3	2	0	Built-up	650	Partly
Kepong	Kuala-Lumpur	200000	3	2	0	Built-up	650	Partly
Kepong	Kuala-Lumpur	185000	3	2	0	Built-up	650	Unfurnished
Kepong	Kuala-Lumpur	105000	3	2	0	Built-up	720	Partly
Kepong	Kuala-Lumpur	150000	3	2	0	Land-area	721	Partly
Kepong	Kuala-Lumpur	170000	3	2	0	Built-up	731	Unfurnished
Kepong	Kuala-Lumpur	218000	2	0	0	Land-area	750	Unfurnished
Kepong	Kuala-Lumpur	300000	3	2	0	Built-up	819	Unfurnished
Kepong	Kuala-Lumpur	218000	3	2	0	Built-up	841	Partly

- Give information if the data searched does not exist in the record.
- YOU ONLY HAVE TO DO SEARCH IN THE COLUMN OTHER THAN AREA AND PRICE.

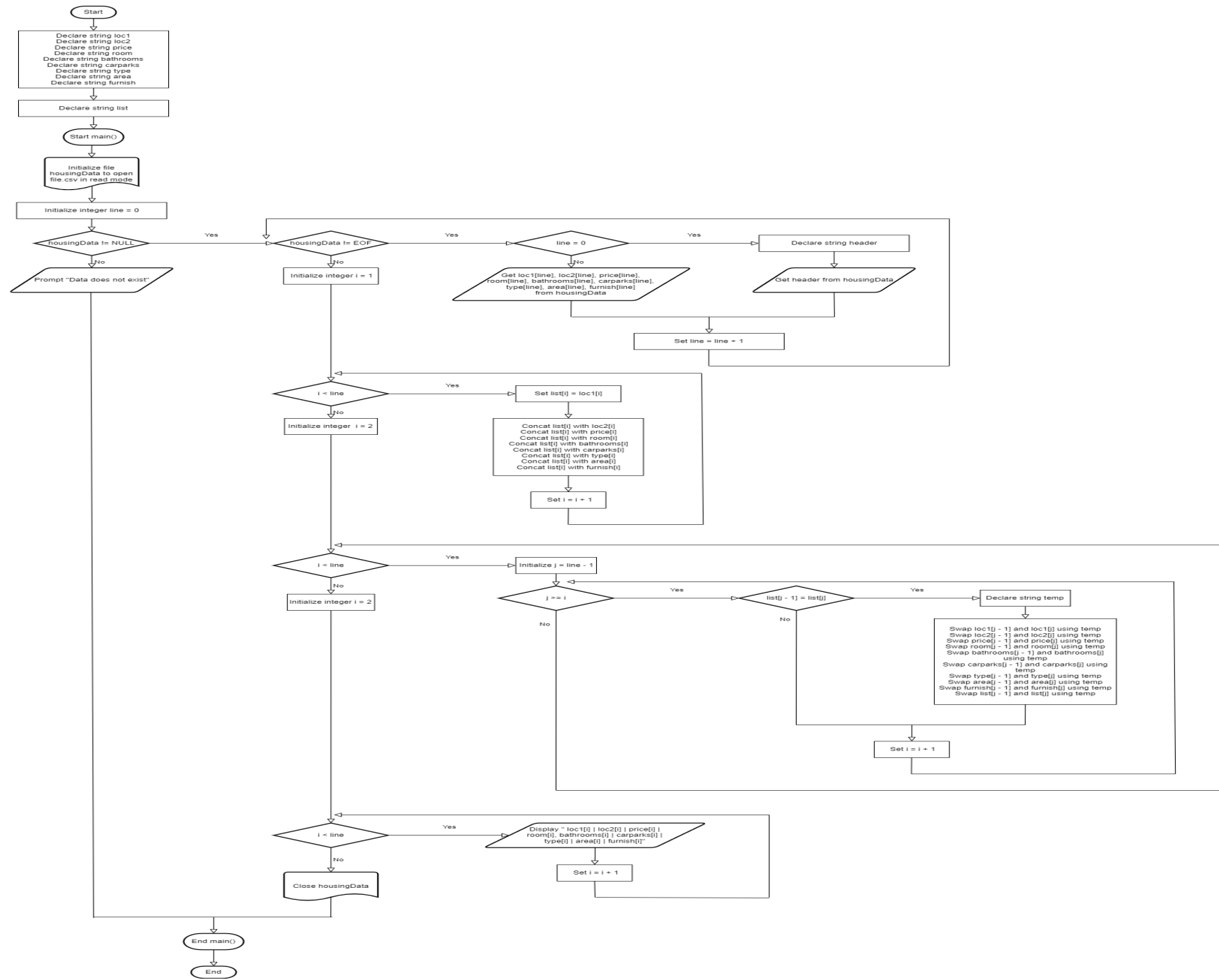
Draw the Flowchart of your solution.

Explanation:



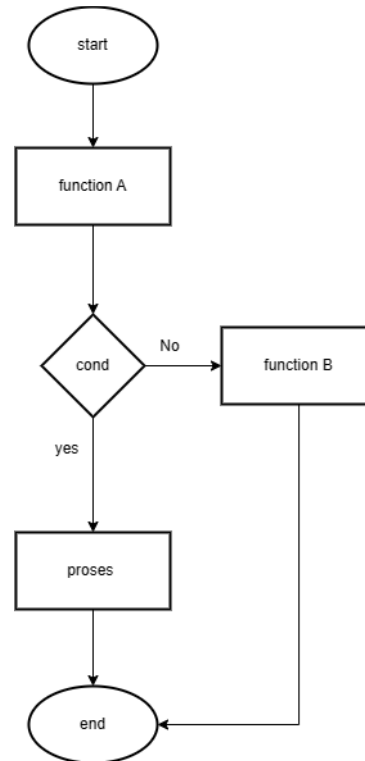
3. **(20 Points)** Please refer to the textbook **Paul J. Deitel (2016). C How to Program: An Introduction to C++, Chapter 8, Exercise 8.21 (page 404). *Alphabetizing a List of Strings***. In the original question, you are asked to alphabetize a list of string using 10 to 15 names of towns. **For this case, implement what is being asked using column Location 1.**
- Draw the Flowchart of your solution.**

Explanation:



Note:

1. All solutions should be built using C code.
2. As mentioned in each question, you also need to draw Flowcharts from your solution for each case number. If your solution contains more than 1 sub program, each subprogram should also be drawn. For example:



Each Function A and Function B must have its own flowchart diagram.